the page or change how the content is parsed. Several directives are supported. The `page` directive denotes this file as representing a navigable page, and it must appear on the top line of the CSHTML file if it represents a page that is intended to be browsed. The `model` directive specifies the data type that acts as a model for this page, which, by default, is the `PageModel` class that accompanies the page. The `PageModel` is the focus of attention later in this chapter.

The next most commonly used directive is likely to be the `using` directive, which brings namespaces into scope, so their types can be referenced without using their fully qualified names. The next listing illustrates the `using static` directive being used to simplify working with the static `Path` class in `System.IO`, which would otherwise conflict with the `Path` property of the Razor page.

Listing 3.2 Adding a using directive with Razor syntax

```
@page
@model WebApplication1.Pages.WelcomeModel
@using static System.IO.Path                          ❶
@{
    var extension = GetExtension("somefile.ext");     ❷
}
```

❶ A using static directive makes the static Path class available without having to specify the class name. Notice there is no semicolon at the end of the using directive as there would be in a C# code file. The semicolon is optional in Razor files.

❷ The static Path.GetExtension method is called without needing to include the class name.

Razor pages support many directives. Some are page-specific, such as the `page` and `model` directives, but others, including the `using` directive, can be applied to multiple pages by including them in a ViewImports file.

The *ViewImports* file, a special kind of file named _ViewImports.cshtml, provides a mechanism for centralizing directives that apply to CSHTML files, so you don't have to add them to Razor pages individually like you did with `System.IO.Path` in the preceding example. The default ViewImports file includes three directives: